```
$ certbot certonly --server http://acme-lan.lan:8000/acme/directory -d db.example.net
```

acme-lan

Real, publicly-trusted certificates for the hosts the internet can never reach.

No relation to the anvil company. Although both are elaborate schemes that mostly work.

PyCon AU 2026 · Brisbane · lightning talk · github.com/smallsam/acme-lan · MIT

1 One name, two worlds

PUBLIC VIEW

`db.example.net`

Not reachable. No open ports. Possibly no A record at all.
The CA cannot knock on this door.

LAN VIEW

`db.example.net → 192.168.3.5`

Split-horizon DNS. A real public domain, pointed at a box that lives behind your firewall forever.

The CA does not need to reach the box. **It only needs proof you control the name.**

That proof is a TXT record. Which means someone has to hold DNS credentials.

```
# dns-01: prove the name, not the network path
```

2 How ACME actually works

A CA will certify any name you can prove you control. Three ways to prove it:

HTTP-01

Serve a token

Put it at /.well-known/acme-challenge/… and the CA fetches it over port 80.

TLS-ALPN-01

Answer a handshake

The token rides inside a special TLS handshake on 443. No HTTP server needed. (RFC 8737)

DNS-01

Publish a TXT record

At _acme-challenge.<name>. The CA looks it up. Nothing of yours has to be reachable at all.

Your LAN host can never answer the first two from the internet. So: dns-01.

```
# the CA never has to see your host — only your proof
```

3 The one prerequisite

Your internal names have to be public names.

NOT NEGOTIABLE

No .local. No .internal. No .lan.

A public CA will not certify a name nobody owns. You need a real, registered zone.

Good time to use that two-letter domain you have been hoarding.

GOOD NEWS

Split horizon stopped hurting

No second copy of the zone to keep in sync. Override only the names you host on the LAN; everything else resolves normally.

```
unbound   local-zone: "example.net." transparent
dnsmasq   address=/db.example.net/192.168.3.5
```

```
# one line per host, and the rest of your zone never notices
```

4 The three usual answers

OPTION A

DNS token on every host

Every box that needs a cert now holds a credential that can rewrite your whole zone.

Including the printer.

OPTION B

Run your own CA

Genuinely fine — until you have to install that root on every laptop, phone, CI runner and contractor.

Forever.

OPTION C

Self-signed and look away

The industry standard.

Every browser warning you have clicked through, and every one you have taught somebody else to click through.

** Yes — you can shrink that token by delegating _acme-challenge to a zone of its own. Hold that thought; we come back to it.*

```
# all three work. none of them made me happy.
```

5 The whole idea

Be an ACME server.

Be an ACME client.

Sit in the middle.

CLIENT DIFF

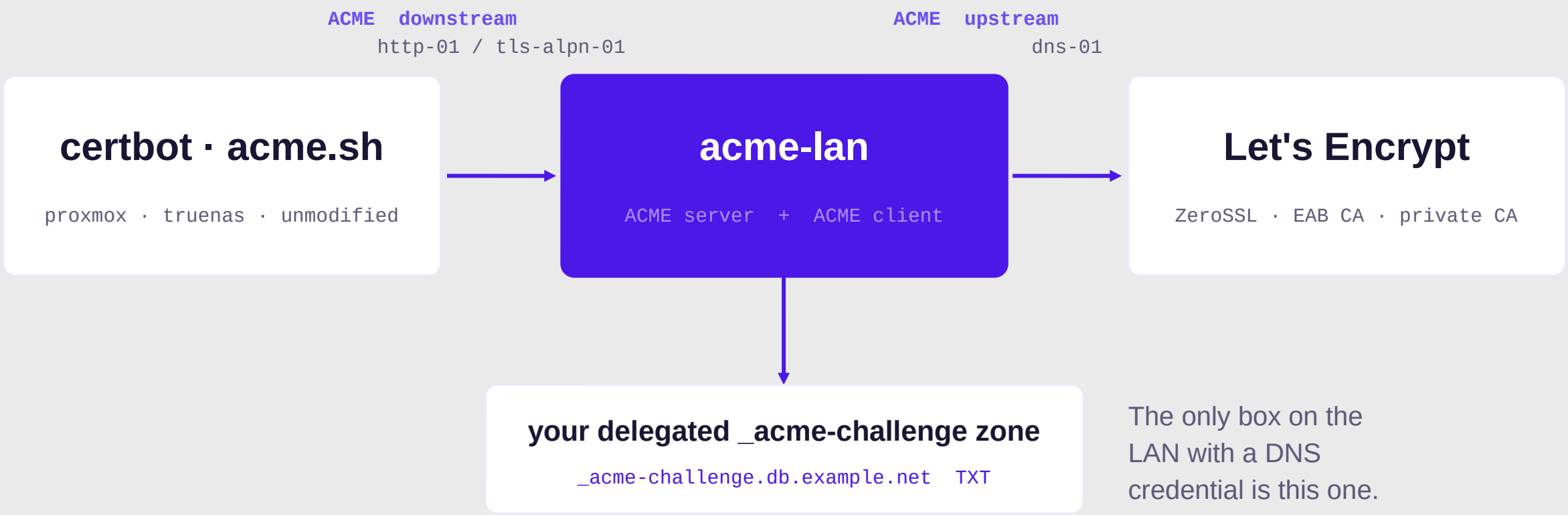
One line.

RFC 8555 both directions, so certbot, acme.sh and the ACME client already built into Proxmox need no plugin, no patch and no idea anything unusual is happening.

```
- --server https://acme-v02.api.letsencrypt.org/directory
+ --server http://acme-lan.lan:8000/acme/directory
```

```
# stock clients only. nothing that needs a custom hook script.
```

6 The architecture, all of it



The only box on the LAN with a DNS credential is this one.

```
# ~5k lines of Python standing between two ACME conversations
```

7 The one decision that matters

acme-lan forwards your CSR upstream.

- The issued cert matches the key the client already has
- acme-lan never sees, stores or transmits that private key
- Renewal is just ACME again — no state to keep in sync

```
order = upstream.new_order(csr_from_the_client)   # not one we made up
```

```
# a proxy that generated its own keys would be a very expensive self-signed cert
```

8 Where the DNS credentials live

OPTION 1

Zone creds, every host

A token that can rewrite the whole zone — your MX, your website — copied onto every box that needs a cert.

Rotating means touching all of them.

OPTION 2

Delegated creds, every host

CNAME _acme-challenge to a zone of its own, so the token can only write there.

Much smaller blast radius. Still on every box, still rotated everywhere.

OPTION 3 · ACME-LAN

Delegated creds, one host

The same narrow token, held in one place. Every other box speaks plain ACME and holds nothing at all.

Rotate it once.

Two axes at once: **what a leaked credential can do, and how many places you have to fix.**

```
# the CNAME is the good idea. one holder for it is the better one.
```

That’s the core.

Everything after this is scope creep I’m happy about —
and the reason I actually still run it.

```
# extras
```

9 For the things that will never run ACME

MODE: DEVICE (PREFERRED)

The key never leaves the box

Fetch a CSR the device generated itself, sign it upstream, push back only the certificate.

Nothing secret ever crosses the wire.

MODE: LOCAL

We make the key and push both

For gear that cannot generate a CSR. The key is stored encrypted — never in plaintext at rest — and the UI tells you off about it anyway.

Install happens through deploy plugins: local (write files, run a reload) and ssh (SFTP + reload).
Most network gear is driven over SSH, so a vendor plugin is one class and a list of PluginFields.

ESXi · iDRAC · iLO · switches · printers · NAS · load balancers

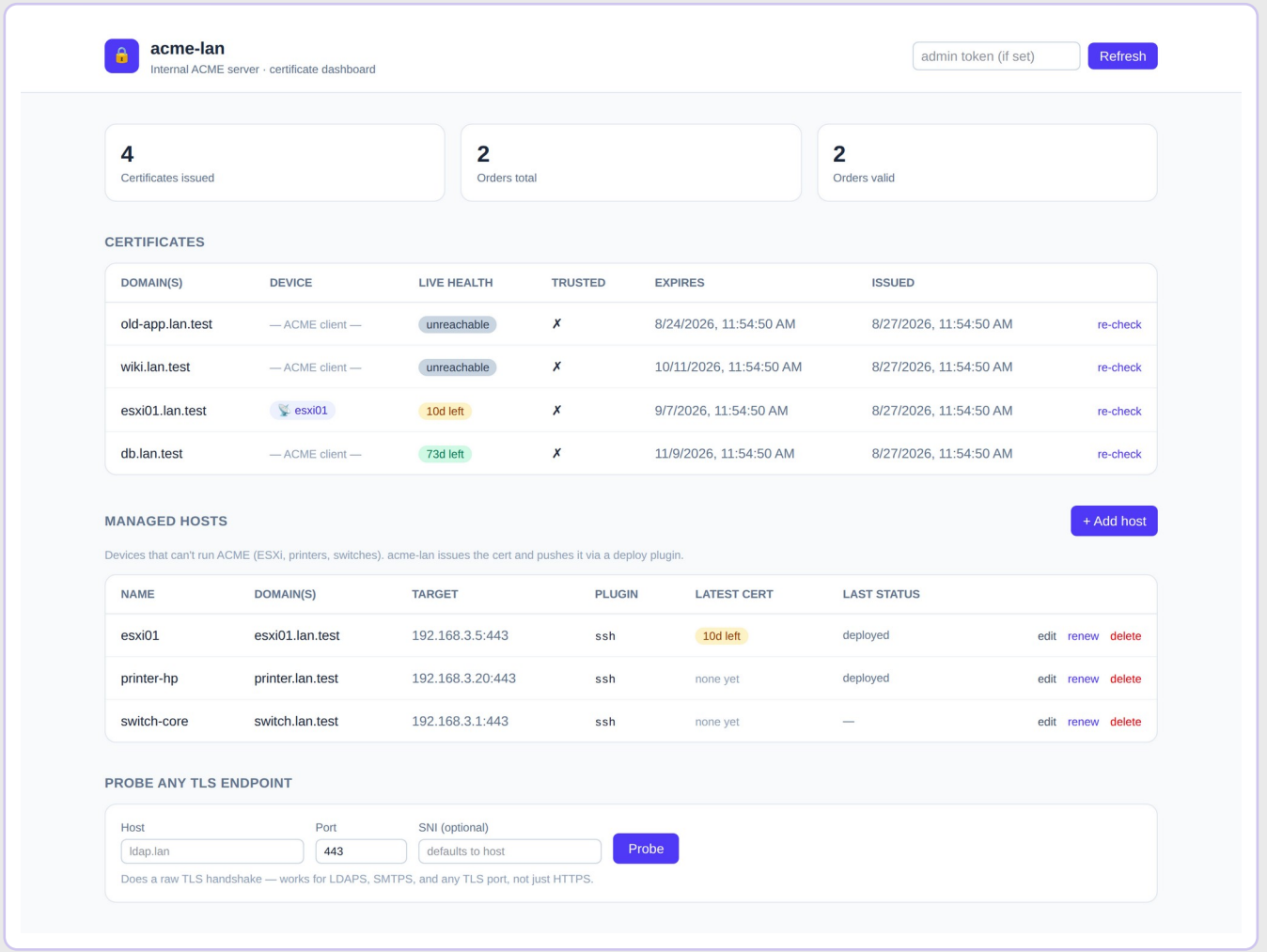
10 It comes with a dashboard

Every certificate, every managed host, one page.

Live health badges are a raw TLS probe, not a database lookup.

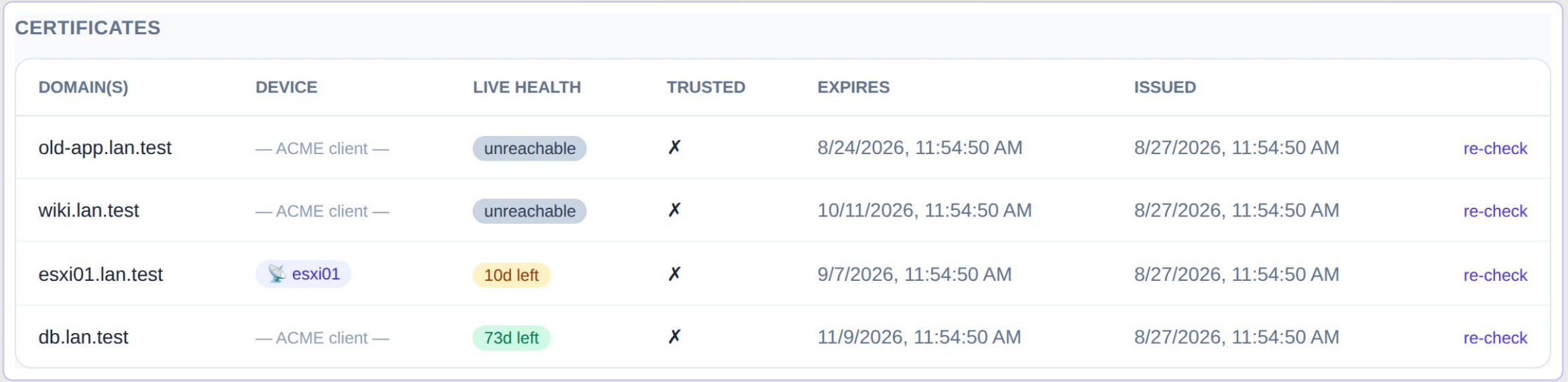
Certificates link back to the device they were pushed to, and back again.

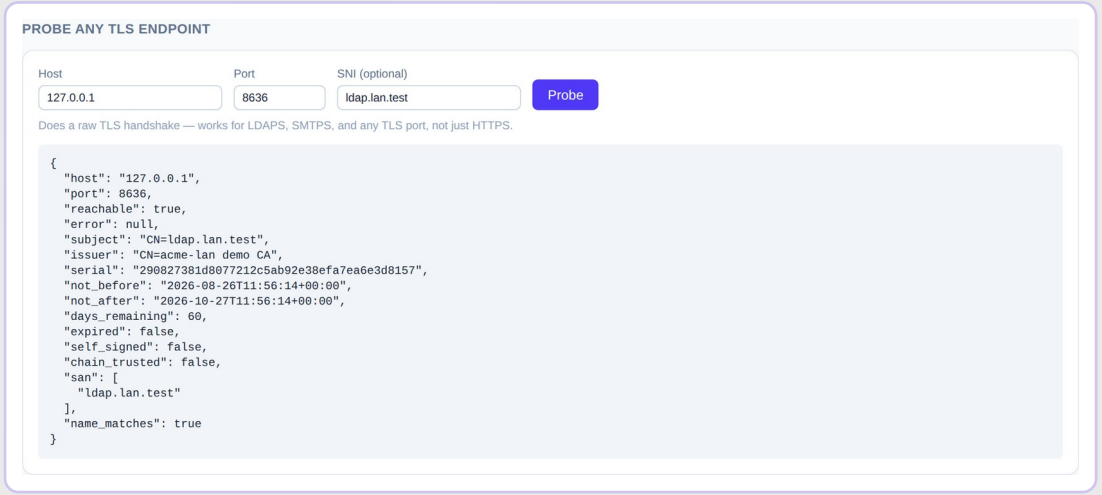
Expiry warnings, retire-when-done, email + webhook notifications.




```
# GET /api/certificates · GET /api/hosts · POST /api/health/probe
```

11 Live health, not a spreadsheet





A raw TLS handshake.

Not an HTTP request that assumes port 443 speaks HTTP.

LDAPS. SMTPS. That one appliance on 8443. Anything that completes a handshake gets an honest answer: expiry, chain trust, SAN match, self-signed.

12 Should you use this? Maybe!

Your hosts reach the internet and can hold DNS creds	**certbot + a DNS plugin. You don’t need me.**
One reverse proxy already fronts everything you own	**Caddy or Traefik. Two lines of config.**
Happy to run a private CA and push a root everywhere	**step-ca / smallstep. Genuinely excellent.**
You want delegated DNS-01 — with the clients you already have	**This. The alternative is standing up acme-dns.**

It is at least easier than the thing I would otherwise be telling you to build.

13 The Python bits

FastAPI + async SQLAlchemy

One app. RFC 8555 endpoints, REST API, and the SPA, all from one process.

certbot's own acme library

The reference implementation is on PyPI. I did not write an ACME client.

cryptography

CSRs, signing, and the raw TLS probe behind every health badge.

Alembic on startup

The container migrates itself. Upgrades are docker pull.

Pebble + pytest

A real ACME CA in a container that issues deliberately terrible certs. e2e for the whole chain.

Vue 3 + Playwright

The dashboard, and UI tests in CI. Sorry — that bit is not Python.

```
# uv sync && uv run acme-lan
```

Try it

```
docker run -d --name acme-lan -p 8000:8000 -v acme-lan-data:/app/data \
  -e ACME_LAN_EXTERNAL_URL="http://acme-lan.example.net:8000" \
  -e ACME_LAN_SECRET_KEY="$(...fernet key...)" \
  -e ACME_LAN_DNS_PROVIDER="cloudflare" \
  ghcr.io/smallsam/acme-lan:latest      # defaults to LE staging
```

github.com/smallsam/acme-lan

MIT · docs for deployment, plugins and key handling in the repo · issues and vendor plugins very welcome

Thanks. No questions — lightning talk rules — but I'm in the hallway,
and my printer finally has a certificate nobody has to click through.

Appendix

Prior art, and the bit about challenge directions that did not fit in five minutes.

```
# for the hallway track
```

Prior art I borrowed from

ACME-DNS

A DNS server with one opinion

Its entire job is answering _acme-challenge, and it expires its own records. Lovely idea — but it is an internet-facing DNS server you run, and clients have to speak acme-dns. Most appliances never will.

acme-lan ships a provider for it anyway.

ACME2CERTIFIER

ACME in front of other CAs

Fronts a CA that is not Let’s Encrypt through ca_handler plugins. That is exactly where acme-lan’s private-CA handler comes from.

But it does not do the split-horizon or device-push half.

Both solve part of it. Neither solved it the way I wanted — so this borrows from both.

Two challenges, two directions

DOWNSTREAM · PROVE IT TO ME

http-01
tls-alpn-01

Port 80 if the box has a web server.

RFC 8737 over port 443 if it does not — the challenge rides in the TLS handshake itself.

Whichever listener the appliance already has.

UPSTREAM · PROVE IT TO THEM

dns-01
http-01 at the edge

dns-01 (default): nothing needs to be publicly reachable. Ever.

edge http-01: faster than DNS propagation, but you need a public IP and a wildcard A record.

One env var switches it.

```
# appendix · ACME_LAN_UPSTREAM_CHALLENGE=dns-01
```